



리눅스 시스템 프로그래밍

1장 리눅스 시스템 프로그래밍 개요

목차

1. 리눅스 커널과 시스템 콜
2. 라이브러리 함수와 시스템 콜
3. 컴파일 옵션과 라이브러리 생성

리눅스 커널과 시스템 콜

1. 리눅스 커널 구조
2. 커널과 시스템 콜의 관계

리눅스 커널과 시스템 콜

- 리눅스 시스템과 네트워크 프로그램이란 리눅스 커널에서 제공하는 기능을 각종 시스템 콜(또는 API)을 이용하여 구현하는 것을 의미한다.
- 따라서 이 프로그램을 작성하기 위해서는 우선 리눅스 커널의 구조를 이해할 필요가 있다. 이번 장에서 리눅스 커널의 구조를 살펴보고 시스템 콜의 호출 과정을 통해 커널과의 관계를 살펴본다.

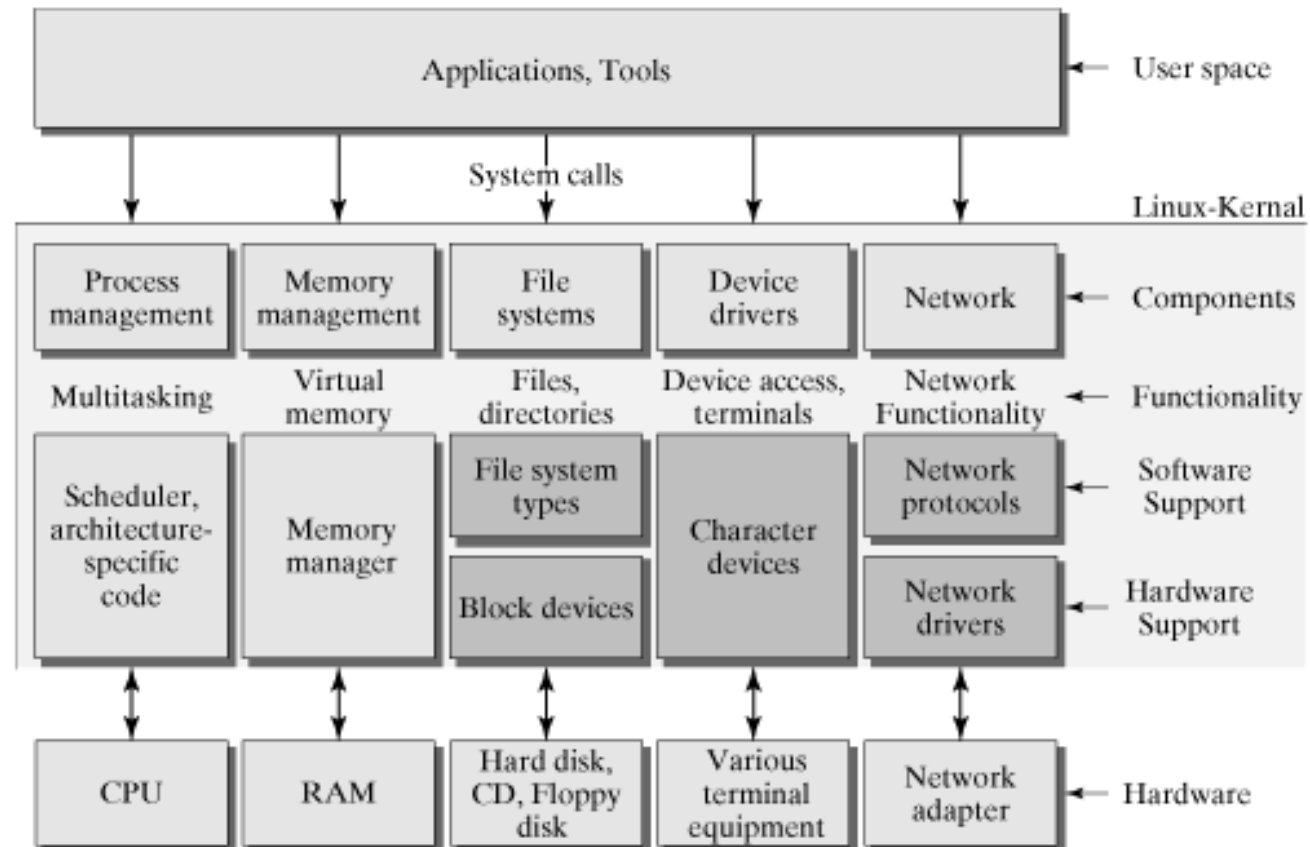
리눅스 커널 구조

- 커널(kernel)이란 컴퓨터 내에 있는 자원들을 동작시키고 관리하여 사용자의 응용 프로그램이 효율적으로 실행될 수 있는 환경을 제공하는 **자원 관리 프로그램**을 말한다.
- 커널 관리 자원
 - H/W 자원(물리적인 자원)

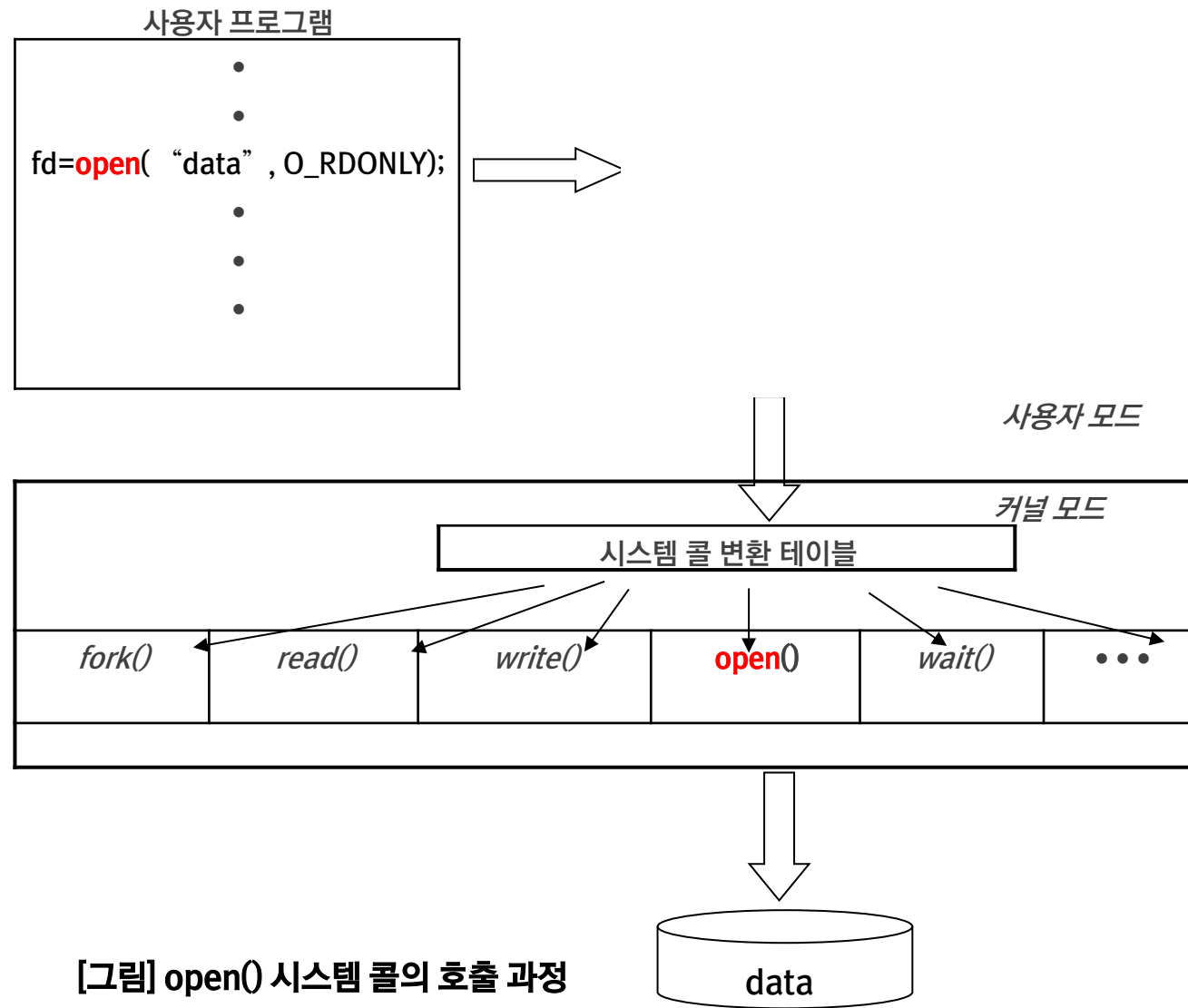
CPU, 메모리, 디스크, 터미널, 네트워크 등 시스템을 구성하고 있는 물리적인 장치
 - S/W 자원(물리적 자원을 추상화한 객체)

프로세스(process)와 스레드(thread), 세그먼트(segment)와 페이지(page),
파일과 아이노드(inode), 통신 프로토콜과 소켓 등
- 커널의 자원을 이용하는 시스템과 네트워크 프로그램을 작성하려면 커널의 구조를 이해하고, 커널에서 관리하는 자원을 어떻게 사용해야 하는지 알아야 한다.

리눅스 커널 구조



[그림] 리눅스 커널 구조 (출처 : <http://flylib.com/books/en/3.475.1.15/1/>)



[그림] open() 시스템 콜의 호출 과정

라이브러리함수와 시스템 콜

- 일반적으로 프로그램을 작성하다 보면 개발자 자신이 직접 함수를 만드는 경우도 있지만 C 또는 C++ 언어에서 제공하는 라이브러리 함수를 이용하는 경우도 많다. 앞서 시스템 콜과 커널의 관계를 살펴본 바에 의하면 시스템 콜도 일종의 함수임을 알게 되었다.

시스템 콜 – 커널이 제공하는 서비스로 응용 프로그램의 요청에 따라 커널에 접근하기 위한 인터페이스 함수를 말한다. 때로 API(Application Programming Interface) 라고도 한다.

라이브러리 함수 – 자주 사용할 만한 기능의 프로그램을 미리 함수로 작성해 놓은 것을 말한다. 라이브러리 함수가 많으면 프로그램의 개발 속도를 높일 수 있다.

- 그럼 라이브러리함수와 시스템 콜에는 어떤 차이점이 있을까? 프로그램 개발 시 주의할 점은 무엇일까?

라이브러리함수와 시스템 콜의 차이

- 일반적으로 시스템 콜이란 커널의 자원을 사용자가 사용할 수 있도록 미리 만들어 놓은 함수들을 말한다. [그림] open 시스템 콜 호출 과정에서 알 수 있듯이 사용자 프로그램이 사용자 모드에서 실행되다가 **시스템 콜을 호출하면 커널 모드로 전환하여 실행**된다.
- 그에 비해 라이브러리 함수란 사용자들이 많이 사용할 것 같은 기능들, 예를 들어 문자열 처리, 표준 입출력, 수학 관련 공식 등을 미리 함수로 만들어 놓은 것으로 이 함수를 호출하여도 사용자 프로그램은 **사용자 모드에서 계속 실행**된다

시스템 콜의 예제 소스 작성

ex01.c

```
#include <stdio.h>
#include <time.h>
#include <sys/types.h>
#include <stdlib.h>

int main(void)
{
    time_t* cur_time;

    time(cur_time);
    printf("Current Time = %d\n", (int) *cur_time);
    return 0;
}
```

위 소스 코드 작성하고 컴파일하여 실행해보자 .
실행 결과가 정상적으로 보이는가?

■ 라이브러리함수의 예제 소스 작성

ex02.c

```
#include <stdio.h>
#include <time.h>
#include <sys/types.h>
#include <stdlib.h>

int main(void)
{
    time_t* cur_time=(time_t*) malloc(sizeof(time_t));

    time(cur_time);
    printf("Current Time = %d\n", (int) *cur_time);

    char* cur_string;

    cur_string=ctime(cur_time);
    printf("Current Time String = %s\n",cur_string);

    return 0;
}
```

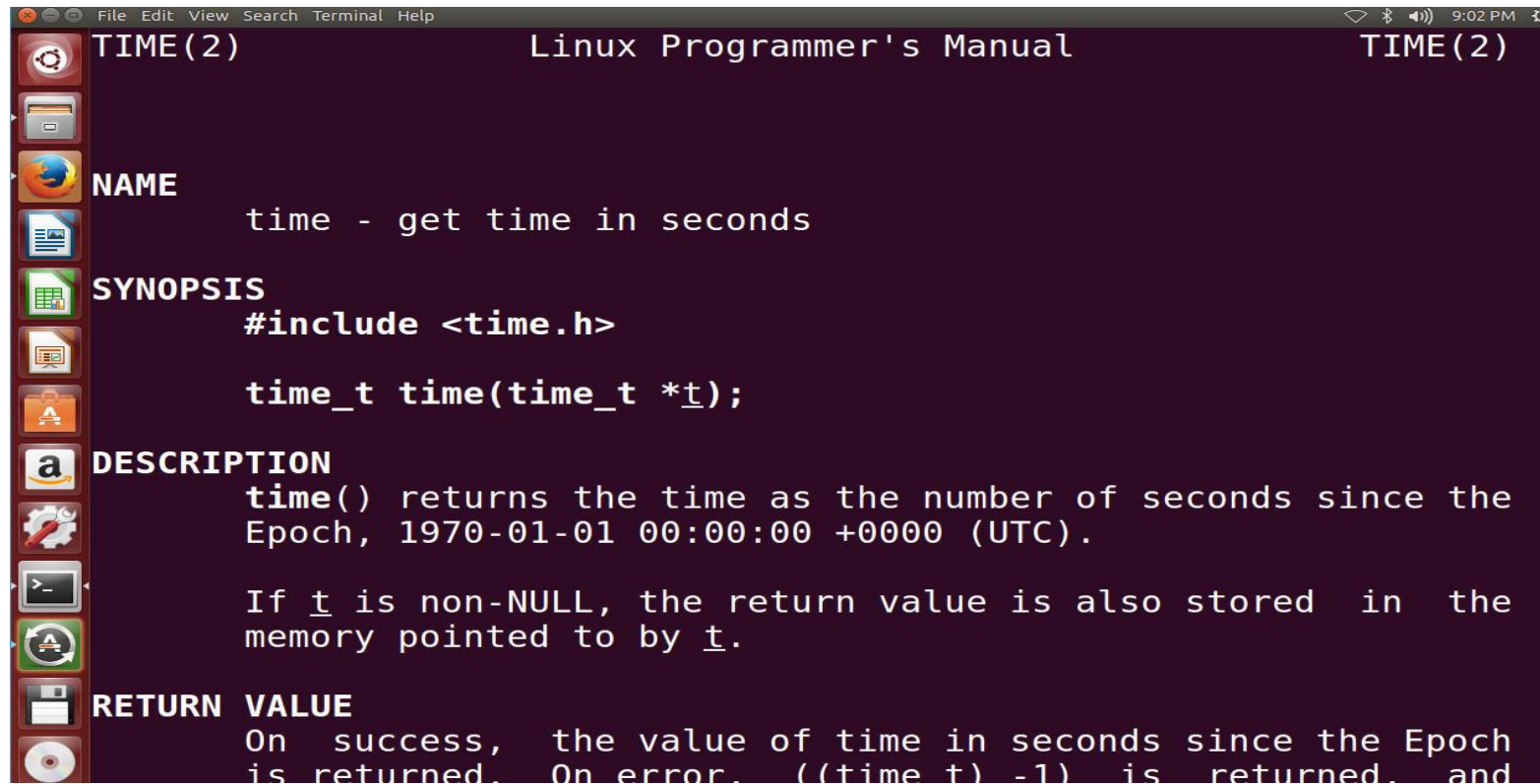
위 소스 코드 작성하고 컴파일하여 실행해보자.
실행 결과가 정상적으로 보이는가?

유의 사항

- 앞서 메모리 할당에 관련된 라이브러리 함수와 시스템 콜의 차이점을 살펴보았다.
- 이외에 리눅스에서 제공하는 온라인 매뉴얼에는 개발자를 위하여 다음과 같은 매뉴얼 섹션을 제공하고 있다. 매뉴얼에 기술되어 있는 내용을 꼼꼼히 살펴 보며 프로그램 개발 시 오류가 생기지 않도록 처리해야 한다.
 - 섹션 2 : 시스템 콜 매뉴얼
 - 섹션 3 : 라이브러리 함수 매뉴얼
- 매뉴얼 사용법은 다음과 같다.
- `man [섹션번호] 라이브러리함수명 또는 시스템콜이름`

유의 사항

- time() 시스템 콜의 매뉴얼 보기
- 수행 명령 : man 2 time



```
TIME(2)                                Linux Programmer's Manual                                TIME(2)

NAME
    time - get time in seconds

SYNOPSIS
    #include <time.h>

    time_t time(time_t *t);

DESCRIPTION
    time() returns the time as the number of seconds since the
    Epoch, 1970-01-01 00:00:00 +0000 (UTC).

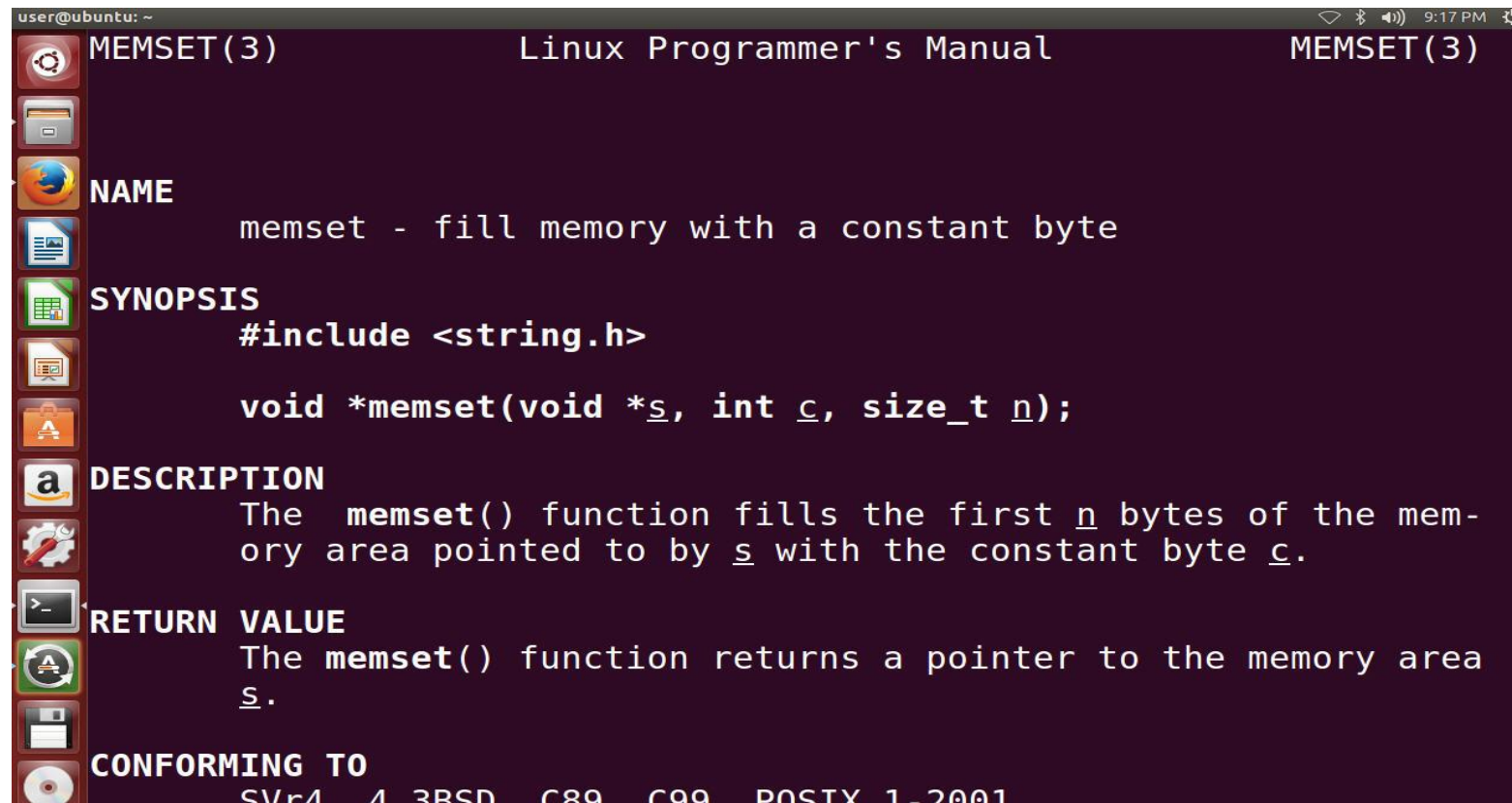
    If t is non-NULL, the return value is also stored in the
    memory pointed to by t.

RETURN VALUE
    On success, the value of time in seconds since the Epoch
    is returned. On error, ((time_t) -1) is returned, and
```

유의 사항

- `memset()` 라이브러리함수의 매뉴얼 보기

수행 명령 : `man 3 memset`



```
user@ubuntu: ~  
MEMSET(3) Linux Programmer's Manual MEMSET(3)  
  
NAME  
memset - fill memory with a constant byte  
  
SYNOPSIS  
#include <string.h>  
  
void *memset(void *s, int c, size_t n);  
  
DESCRIPTION  
The memset() function fills the first n bytes of the mem-  
ory area pointed to by s with the constant byte c.  
  
RETURN VALUE  
The memset() function returns a pointer to the memory area  
s.  
  
CONFORMING TO  
SVr4, 4.3BSD, C89, C99, POSIX 1-2001
```

유의 사항

- 라이브러리함수와 시스템 콜의 차이점

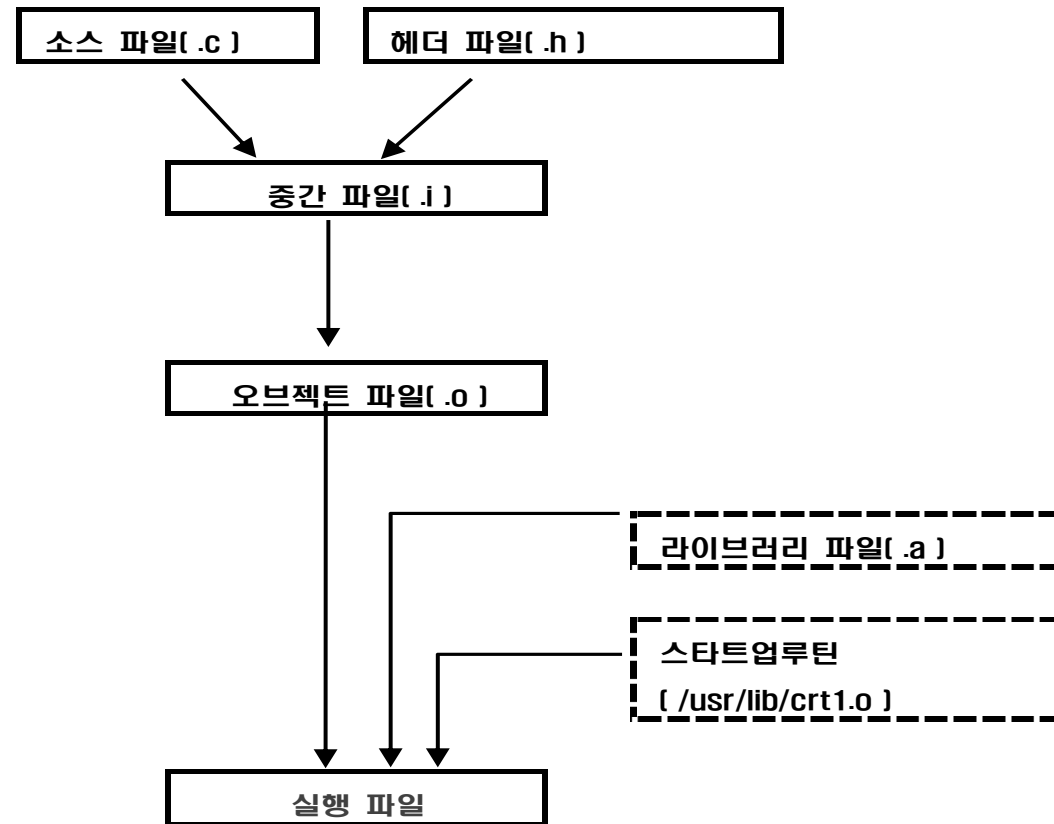
구분	시스템 콜	라이브러리 함수
매뉴얼 참조	man 2 시스템콜명	man 3 라이브러리함수명
수행공간	커널 모드	사용자 모드
메모리할당여부	별도로 사용자 모드에서 메모리 할당 필요.	라이브러리 함수에서 할당된 메모리 이용 가능

컴파일 옵션과 라이브러리 생성

1. 컴파일 과정과 옵션
2. 라이브러리 생성

컴파일 과정과 옵션

■ 컴파일 과정



컴파일 과정과 옵션

■ 전처리 과정

-**D***name*: *name*에 지정된 심볼을 1로 정의한다. 조건부 컴파일에서 유용하게 사용

-**I** : 헤더 파일의 위치를 지정하는 옵션으로 표준 디렉토리 (/usr/include) 이외
에 있는

헤더 파일의 디렉토리 경로를 지정

■ 문법 검사 과정

-**c** : 컴파일 단계까지만 수행하고, 링크를 수행하지 않은 상태에서 오브젝트 파

일(.o)을 생성

-**O***N* : 컴파일된 코드를 최적화 시키는 옵션으로 최적화 단계를 *N*으로 지정

($0 \leq N \leq 3$)

■ 링킹 과정

-**L** : 라이브러리 파일이 존재하는 디렉토리 경로를 지정

-**l** *string*: 옵션 뒤에 지정한 라이브러리 파일을 컴파일 시에 링크

단 라이브러리 파일명은 lib*string*.a 또는 lib*string*.so

■ 기타 유용한 옵션

-**g** : 실행 파일에 표준 디버깅 정보를 포함되므로 gdb, DDD 등의 디버거를

이용하려고 할 때 이 옵션을 이용하여 컴파일해야 한다.

-**Wall**: gcc가 제공하는 모든 경고를 나타낸다. 시스템과 네트워크 프로그램이나

커널 기반의 프로그램 작성시 가급적 이 옵션을 함께 넣어 컴파일하는 것

이 좋다.

■ 세부 내용은 컴파일러의 매뉴얼 참조

컴파일 옵션의 사용 예

ex03.c

```
#include <stdio.h>
#include <math.h>

int main(void)
{
    double dnum = 100.0;

    printf( "SQRT(%lf)= %lf\n", dnum, sqrt(dnum));
    return 0;
}
```

위 소스 코드를 특별한 옵션을 붙이지 말고 컴파일 해봅시다.

컴파일이 정상적으로 이루어지는가?

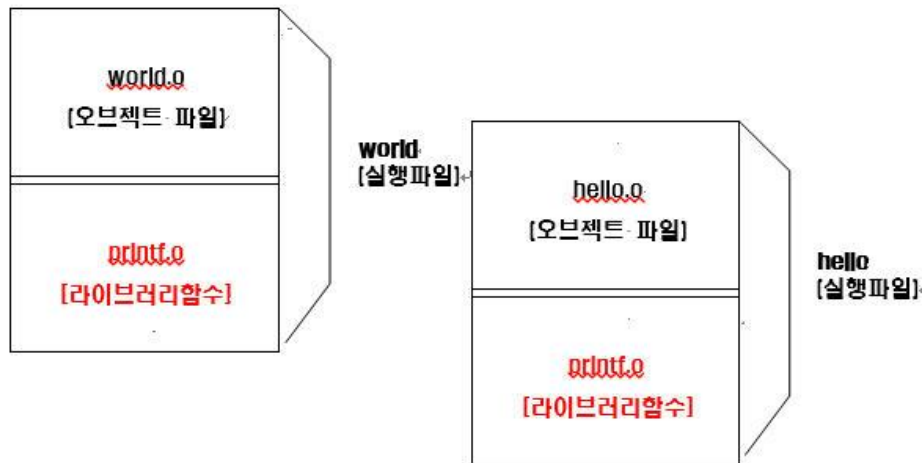
라이브러리 생성

- 프로젝트를 진행할 때 여러 사람들이 함께 개발하는 경우가 종종 있다. 그 때 같은 기능을 여러 사람이 사용하게 되는 경우 이를 개별적으로 작성하여 각각의 소스 프로그램에 함수로 만드는 방법보다는 라이브러리를 만들어 함께 사용하면 훨씬 편리하고 동일한 기능이 소스 코드 여러 군데 중복되지 않아 효율적인 프로그램을 작성할 수 있으며 다른 프로젝트에서도 활용할 수 있다.
- 기본적으로 제공되는 표준 라이브러리는 /usr/lib에 존재하며, C 컴파일러는 프로그램 컴파일 시 기본적으로 이 디렉토리에서 라이브러리 함수를 찾아 링크하게 된다. gcc 의 '-L' 과 '-l' 옵션을 이용하면 다른 디렉토리에 있는 라이브러리도 이용할 수 있다. 라이브러리 파일의 이름은 lib로 시작되며 라이브러리의 의미를 나타내는 이름 다음에 파일의 확장자 .a 나 .so 로 구성된다.
- 라이브러리에는 정적(static) 라이브러리와 공유(shared) 라이브러리 두 종류가 있는데 이에 대하여 그 차이점과 생성 방법을 알아보기로 하자

라이브러리 생성

- 정적 라이브러리.

프로그램의 오브젝트 코드 + 미리 만들어 놓은 라이브러리 함수의 오브젝트 코드 = 실행 파일



이 world 와 hello 실행 파일을 동시에 실행한다면 ?

라이브러리 생성

- 공유 라이브러리

프로그램의 오브젝트 코드 + 라이브러리함수에 관한 정보 = 실행 파일



이 world 와 hello 실행 파일을 동시에 실행한다면 ?

라이브러리 생성

- 라이브러리 작성 방법

- 정적 라이브러리

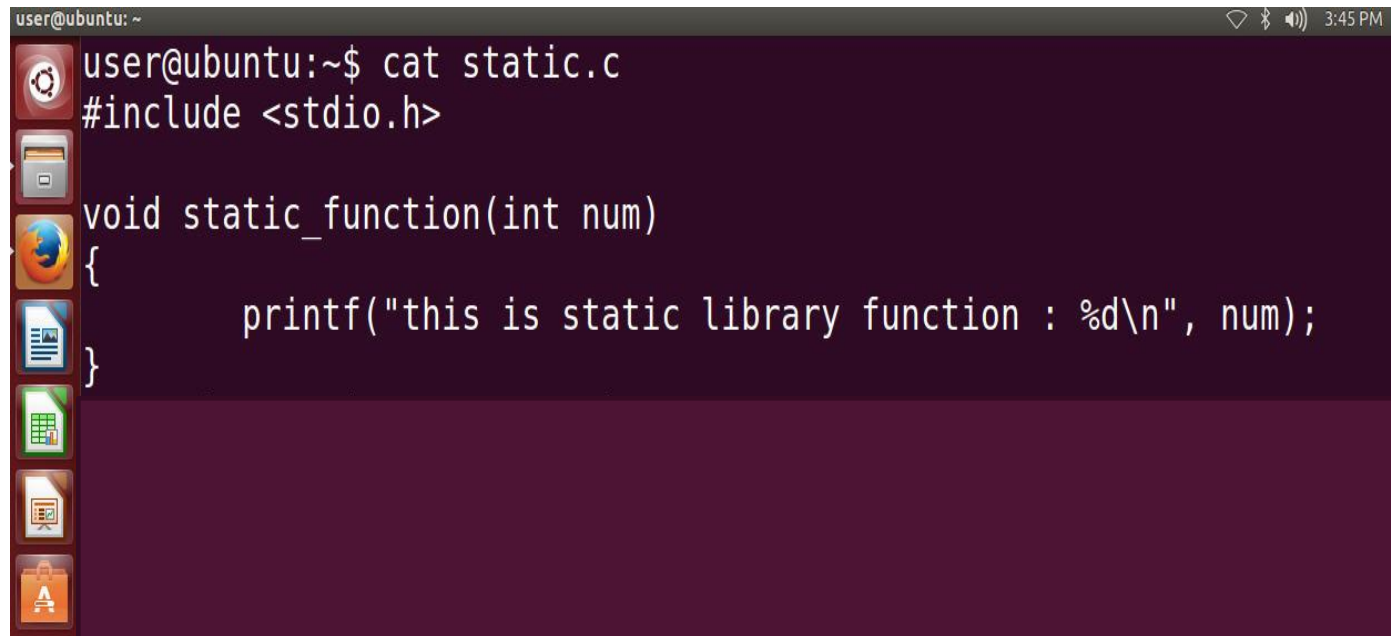
작성 절차	사용 예
소스 파일 생성	vi static.c
오브젝트 파일 생성	gcc -c static.c
라이브러리 파일 생성	ar rv lib##.a static.o

- 공유라이브러리

작성 절차	사용 예
소스 파일 생성	vi shared.c
오브젝트 파일 생성	gcc -c -fPIC shared.c
라이브러리 파일 생성	ld -shared -o lib##.so shared.o

라이브러리 생성

- 작성 예
- 정적 라이브러리



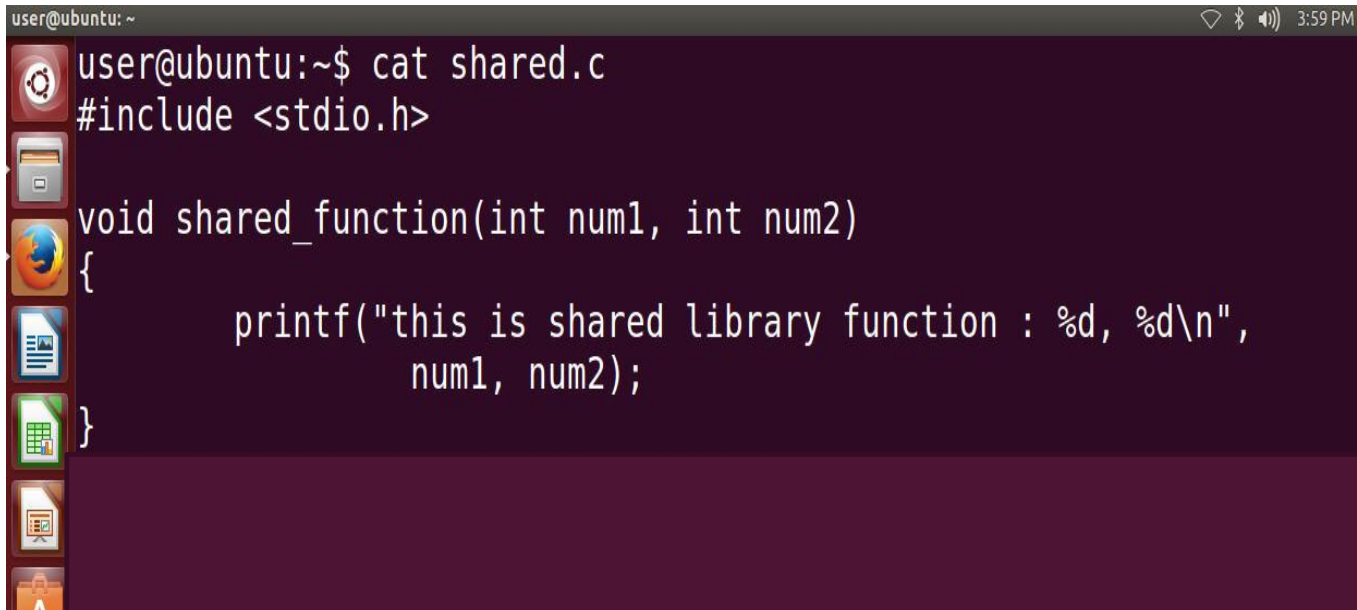
A terminal window titled 'user@ubuntu: ~' with a system clock of 3:45 PM. The window shows the command 'cat static.c' and the contents of the file 'static.c'. The file contains a C function definition for 'static_function' that takes an integer 'num' and prints a message. The terminal has a dark background and a vertical sidebar on the left with various application icons.

```
user@ubuntu:~$ cat static.c
#include <stdio.h>

void static_function(int num)
{
    printf("this is static library function : %d\n", num);
}
```


라이브러리 생성

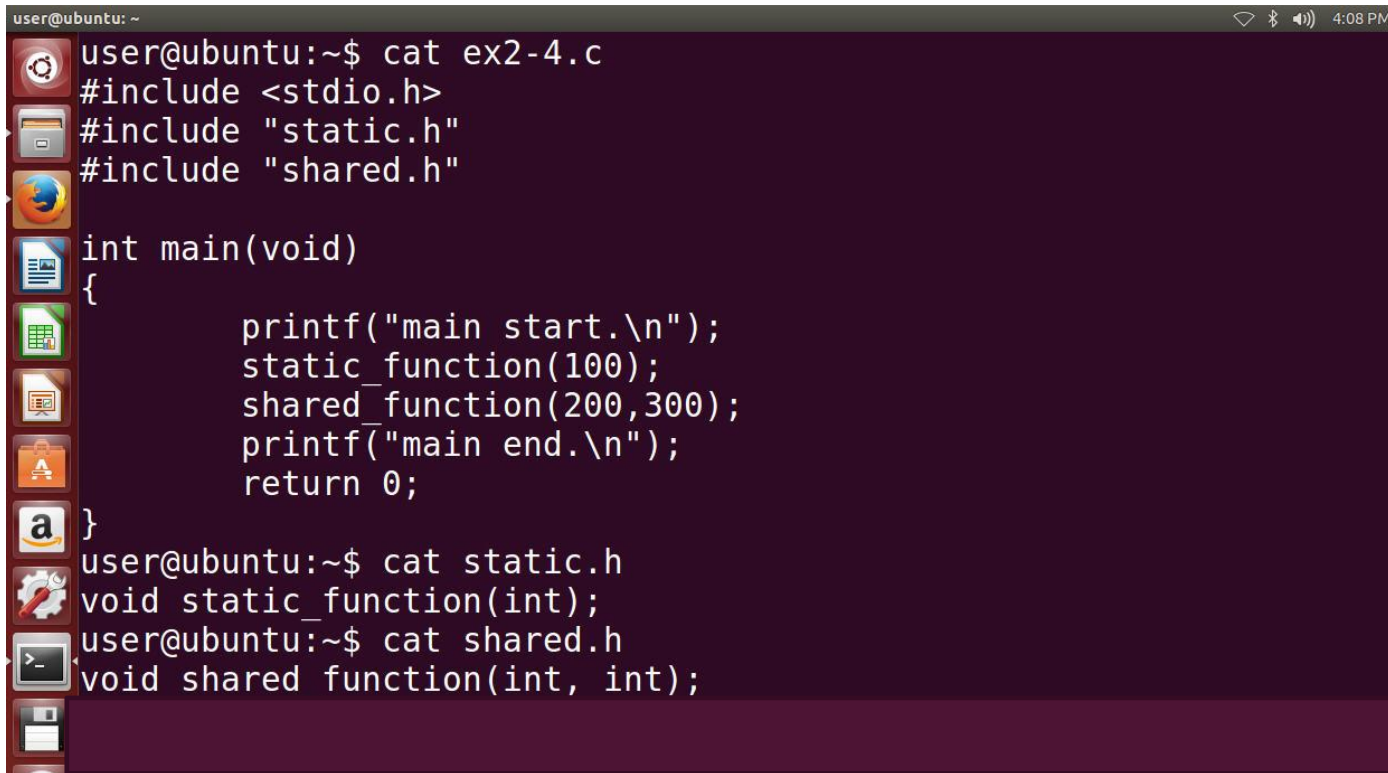
- 작성 예
- 공유 라이브러리



```
user@ubuntu: ~  
user@ubuntu:~$ cat shared.c  
#include <stdio.h>  
  
void shared_function(int num1, int num2)  
{  
    printf("this is shared library function : %d, %d\n",  
          num1, num2);  
}
```

라이브러리 생성

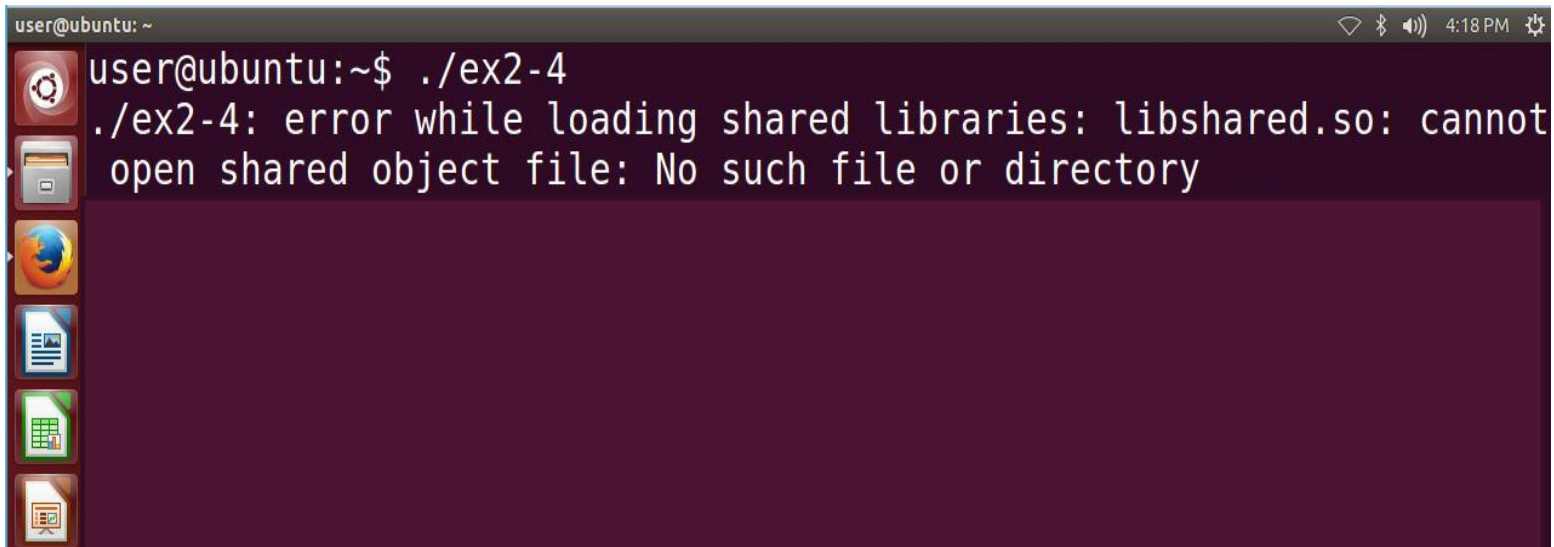
- 작성 예
- 응용프로그램에서의 사용 예-1



```
user@ubuntu: ~  
user@ubuntu:~$ cat ex2-4.c  
#include <stdio.h>  
#include "static.h"  
#include "shared.h"  
  
int main(void)  
{  
    printf("main start.\n");  
    static_function(100);  
    shared_function(200,300);  
    printf("main end.\n");  
    return 0;  
}  
user@ubuntu:~$ cat static.h  
void static_function(int);  
user@ubuntu:~$ cat shared.h  
void shared_function(int, int);
```

라이브러리 생성

- 작성 예
- 응용프로그램에서의 사용 예-2

A terminal window from Ubuntu showing a command execution and an error. The window title is 'user@ubuntu: ~'. The command entered is './ex2-4'. The output is an error message: './ex2-4: error while loading shared libraries: libshared.so: cannot open shared object file: No such file or directory'. The terminal has a dark background with light text. On the left side of the terminal window, there is a vertical dock with several application icons: a gear (system settings), a folder (file manager), a globe (web browser), a document with a magnifying glass (text editor), a spreadsheet (calculator), and a presentation slide (powerpoint). The top right corner of the terminal window shows system status icons: a Wi-Fi signal, a Bluetooth icon, a speaker icon, and the time '4:18 PM'.

〈실습〉

- 리눅스 시스템에서 제공하는 `sysinfo()` 함수를 이용하여 리눅스 시스템의 통계정보를 확인하는 프로그램 (`exercise.c`)을 작성하시오.
- 부팅 이후 시스템의 생존 시간(초단위)
- 1분, 5분, 15분 이전의 시스템 평균 부하
- 메모리의 총 크기 / 사용 가능한 메모리
- 스왑 공간의 총 크기 / 사용 가능한 스왑 공간
- 현재 수행되고 있는 프로세스 수



리눅스 시스템 프로그래밍

2장 리눅스 시스템 프로그래밍 기본

목차

1. 오류처리와 환경변수
2. 사용자와 프로세스 정보
3. 호스트와 시간 정보
4. 시스템 자원의 제한치

오류 처리와 환경 변수

1. 오류 처리
2. 환경 변수

오류 처리

■ errno 변수

- 리눅스 시스템에서는 시스템 콜 및 라이브러리 함수를 수행하다가 오류가 발생하면 사용자 프로그램으로 다음과 같은 값을 반환한다.

시스템 콜 오류 시 리턴 값 : -1

라이브러리 함수 오류 시 리턴 값 : NULL

- 이때 `errno` 변수에 오류의 원인을 확인할 수 있는 구체적인 값이 설정된다. 이 변수는 `errno.h` 헤더 파일에 다음과 같이 선언되어 있다.

```
extern int errno;
```

```
extern const char * sys_errlist[];  
extern int sys_nerr;
```

- 이 `errno` 변수에 설정될 수 있는 값들은 `/usr/include/asm`

```
#define EPERM      1    /* Operation not permitted */  
#define ENOENT     2    /* No such file or directory */  
#define ESRCH      3    /* No such process */  
#define EINTR      4    /* Interrupted system call */  
#define EIO        5    /* I/O error */  
.....  
#define EREMOTEIO  121   /* Remote I/O error */  
#define EDQUOT     122   /* Quota exceeded */  
#define ENOMEDIUM  123   /* No medium found */  
#define EMEDIUMTYPE 124   /* Wrong medium type */
```


오류 처리

■ errno 변수의 설정 값

- 시스템 콜 및 라이브러리 함수의 리턴 값이나 errno 변수에 설정되는 값들은 리눅스의 온라인 매뉴얼(man 명령)을 이용하여 자세히 알아볼 수 있다.
- 다음은 close 시스템 콜의 매뉴얼이다.

```
user@ubuntu: ~/online_lsp/ch04
CLOSE(2)                                Linux Programmer's Manual                                CLOSE(2)

NAME
    close - close a file descriptor

SYNOPSIS
    #include <unistd.h>

    int close(int fd);

DESCRIPTION
    close() closes a file descriptor, so that it no longer refers to any file and may be reused. Any record locks (see fcntl(2)) held on the file it was associated with, and owned by the process, are removed (regardless of the file descriptor that was used to obtain the lock).

    If fd is the last file descriptor referring to the underlying open file description (see open(2)), the resources associated with the open file description are freed; if the descriptor was the last reference to a file which has been removed using unlink(2) the file is deleted.

RETURN VALUE
    close() returns zero on success. On error, -1 is returned, and errno is set appropriately.

ERRORS
    EBADF  fd isn't a valid open file descriptor.

    EINTR  The close() call was interrupted by a signal; see signal(7).

    EIO    An I/O error occurred.

CONFORMING TO
    SVr4, 4.3BSD, POSIX.1-2001.

NOTES
    Manual page close(2) line 1 (press h for help or q to quit)
```

오류 처리

- 오류 메시지 출력
- 오류 메시지를 출력할 때 가장 널리 사용되는 함수는 `perror()`로 그 형식은 다음과 같다.

```
#include <stdio.h>
void perror(const char *string);
```

여기에서 `string`에는 사용자가 출력하려는 메시지를 전달한다.

다음 프로그램을 작성하고 실행해 보시다.

소스코드 (ex01.c)

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(void)
{
    int fd=3;

    if(close(fd)==-1){
        perror("close");
        exit(1);
    }
    return 0;
}
```

환경변수

■ 환경변수란?

리눅스에서는 셸(bash)을 통해 명령 및 프로그램을 실행하는데 셸에는 환경 변수가 있어 사용자의 명령이나 프로그램을 수행하는데 필요한 정보를 가지고 있다. 예를 들어 PATH 환경 변수에는 리눅스 명령에 대한 실행 파일 위치가 포함되어있어 사용자로부터 명령이 입력되면 이 PATH 변수에 등록된 경로에 따라 실행 파일을 찾아내어 커널로 실행을 요청한다. 따라서 PATH 변수의 내용이 잘못되면 명령을 수행할 수 없게 된다. 이와 같이 셸의 동작 즉, 사용자의 명령 및 프로그램 수행과 관련된 환경변수에는 여러 가지가 있다. 다음은 셸의 환경 변수와 그 설정 값들을 확인하는 명령을 수행한 화면이다.

```
user@ubuntu: ~  
user@ubuntu:~$ set | more  
BASH=/bin/bash  
BASHOPTS=checkwinsize:cmdhist:complete_fullquote:expand_aliases:extglob:extquote  
:force_fignore:histappend:interactive_comments:progcomp:promptvars:sourcepath  
BASH_ALIASES=()  
BASH_ARGC=()  
BASH_ARGV=()  
BASH_CMDS=()  
BASH_COMPLETION_COMPAT_DIR=/etc/bash_completion.d  
BASH_LINENO=()  
BASH_SOURCE=()  
BASH_VERSIONINFO=([0]="4" [1]="3" [2]="11" [3]="1" [4]="release" [5]="x86_64-pc-lin  
ux-gnu")  
BASH_VERSION='4.3.11(1)-release'  
CLUTTER_IM_MODULE=xim  
COLORTERM=gnome-terminal  
COLUMNS=80  
COMPIZ_BIN_PATH=/usr/bin/  
COMPIZ_CONFIG_PROFILE=ubuntu  
DBUS_SESSION_BUS_ADDRESS=unix:abstract=/tmp/dbus-HsAJvkTE8L  
DEFAULTS_PATH=/usr/share/gconf/ubuntu.default.path  
DESKTOP_SESSION=ubuntu  
DIRSTACK=()  
DISPLAY=:0
```

환경변수

■ 셸의 환경 변수 중 대표적인 변수

환경 변수	의미
HOME	현재 로그인 사용자의 홈 디렉토리
PATH	명령을 검색할 디렉토리 목록(: 으로 구분)
PS1	셸 프롬프트
MAIL	수신된 메일의 위치
MANPATH	온라인 매뉴얼(man) 검색 디렉토리 목록(: 으로 구분)
USERNAME	로그인 사용자 명

■ getenv(), putenv() 함수

- 리눅스 프로그램에서 셸의 환경 변수를 이용하는 경우는 어떤 경우 일까?
- 예를 들어 프로그램이 실행될 때 사용자의 홈 디렉토리에 test.log 파일을 생성해야 한다고 가정해보자. 홈 디렉토리는 사용자 계정마다 모두 다르므로 디렉토리명을 지정할 수 없다. 이 때 사용자의 홈디렉토리를 저장하고 있는 셸의 환경 변수인 HOME 을 이용하면 쉽게 해결할 수 있다.

환경 변수

- 환경 변수를 이용할 수 있는 함수

```
#include <stdlib.h>
char *getenv(const char *name);
int putenv(const char *string);
```

- 여기에서 `getenv()` 함수는 환경 변수의 이름을 인수로 전달하면 그 변수에 설정되어있는 값을 반환해준다. `putenv()` 함수는 ‘변수명=값’ 형식의 인수를 전달하면 환경 변수의 값이 변경된다. 단, 이 환경 변수의 변경은 해당 프로세스에만 적용되고 다른 프로세스에는 영향을 미치지 않는다.
- 이 함수들을 이용하여 앞에서 명시한대로 사용자의 홈 디렉토리에 `test.log` 파일을 생성하는 프로그램을 만들어보자.

홈 디렉토리에 test.log파일을 생성하도록 빈 칸을 완성해봅시다.

소스코드 (ex02.c)

```
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    char *homedir, filename[80];
    FILE *fp;

    if((fp=fopen(filename, "w"))==NULL) {
        perror("fopen");
        exit(1);
    }
    fwrite("getenv_test\n", 12, 1, fp);
    fclose(fp);
    return 0
}
```

사용자와 프로세스 정보

1. 사용자 정보
2. 프로세스 정보

사용자 정보

■ getuid(), getlogin()

프로그램이 수행되는 동안 현재 이 프로그램을 수행시키는 사용자가 누구인지, 그 사용자의 홈 디렉토리는 어디인지 등 사용자와 관련된 정보가 필요한 경우가 있다. 이런 경우 리눅스에서는 다음과 같은 함수를 이용하여 현재 프로그램을 수행 중인 사용자를 확인할 수 있다.

```
#include <sys/types.h>
#include <unistd.h>
```

```
uid_t getuid(void);
char *getlogin(void);
```

UID(User Identifier) : 사용자 고유의 번호, 리눅스에서 사용자 관리를 위한 고유 번호를 의미

- 여기서 getuid() 함수는 UID를, getlogin() 함수는 사용자 계정의 이름을 반환한다.
- 다음의 사용 예를 살펴보자.

소스코드 (ex03.c)

```
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>

int main(void)
{
    printf("UID = %d\n", getuid());
    printf("Name = %s\n", getlogin());
    return 0;
}
```


사용자 정보

- `getpwuid()`, `getpwnam()` 함수

- 리눅스에서 사용자 계정의 정보를 보관하는 파일은 `/etc/passwd` 이다. 이 파일에는 계정명 이외에 UID, GID, 홈 디렉토리, 로그인 셸 등 상세한 정보가 포함되어 있다. 이와 같은 사용자에 대한 상세한 정보를 확인하는 함수에는 다음과 같은 함수들이 있다.

```
#include <sys/types.h>
```

```
struct passwd *getpwuid(uid_t uid);  
struct passwd *getpwnam(const char *name);
```

GID (Group Identifier) : 사용자가 다른 사용자와 자원을 공유할 수 있도록 연결시키는 이름 (예를 들면, 프로젝트 이름이나 부서 이름 등)의 식별자로 한 사용자는 한 개 이상의 그룹 구성원이 될 수 있으므로, 여러 개의 GID를 가질 수도 있다. 리눅스 사용자라면 누구라도 사용자ID와 그룹ID를 가진다.

홈 디렉토리 : 사용자가 로그인할 때 처음 진입하는 디렉토리를 의미한다.

- `getpwuid()` 함수에는 UID를, `getpwnam()` 함수에는 사용자명을 인수로 전달하면, 사용자에 대한 상세 정보를 다음과 같은 `struct passwd` 구조체 자료형으로 반환한다.

```
struct passwd {  
    char *pw_name;           // 로그인 사용자명  
    uid_t pw_uid;            // UID  
    gid_t pw_gid;            // GID  
    char *pw_dir;            // 홈 디렉토리  
    char *pw_gecos;          // 주석  
    char *pw_shell;          // 로그인 셸  
};
```

ex04.c 를 실행하여 사용자 정보를 확인해봅시다.

```
#include <stdio.h>
#include <sys/types.h>
#include <pwd.h>
#include <unistd.h>

int main(void)
{
    struct passwd *userinfo;

    userinfo = getpwuid(getuid());
    printf("getpwuid() :\nname = %s, uid = %d, gid = %d, home directory = %s, shell = %s\n",
        userinfo->pw_name, userinfo->pw_uid, userinfo->pw_gid,
        userinfo->pw_dir, userinfo->pw_shell);

    userinfo = getpwnam(getlogin());
    printf("\ngetpwnam() :\nname = %s, uid = %d, gid = %d, home directory = %s, shell = %s\n",
        userinfo->pw_name, userinfo->pw_uid, userinfo->pw_gid,
        userinfo->pw_dir, userinfo->pw_shell);

    return 0;
}
```

사용자 함수

- 기타 사용자 관련 함수
- 리눅스에는 앞서 소개한 함수 이외에 사용자 관련 함수로 다음과 같은 함수를 제공한다.

```
#include <sys/types.h>
#include <unistd.h>

uid_t geteuid(void);           // effective UID 알아보기
gid_t getgid(void);           // GID 알아보기
gid_t getegid(void);          // effective GID 알아보기
int setuid(uid_t uid);         // UID 변경
int setgid(gid_t gid);         // GID 변경
```

- 리눅스 사용자 ID, 즉 UID 에는 다음과 같은 2 종류가 있다.

Real UID – 실제로 로그인 한 사용자 계정의 UID

Effective UID – 파일 접근의 효과를 가지고 있는 UID

- 일반적으로 로그인하면 real UID와 effective UID 가 동일하나, set-UID 퍼미션이 설정되어 있는 프로그램을 수행하면 일시적으로 effective UID가 변경된다. 이 effective UID는 파일 접근 시 적용되는 UID로 파일에 대한 읽기, 쓰기, 실행하기 의 권한을 결정한다. 그룹 권한도 사용자의 권한과 동일하다. 이와 같은 effective UID와 GID 를 알아보는 함수는 위에서 소개한 `geteuid()`, `getegid()` 함수이다. 또한 UID 및 GID 를 변경할 수도 있는데 `setuid()`, `setgid()` 함수가 바로 그 함수이다. 이 두 함수는 오로지 root 계정만 사용할 수 있다.

호스트와 시간 정보

1. 호스트 정보
2. 시간 정보

호스트 정보

- `uname()`, `gethostname()`
- 프로그램을 작성하다 보면 시스템의 이름이나 OS 종류 및 버전 등에 대한 정보가 필요한 경우가 있다. 이런 경우 `uname()` 함수를 통해 시스템의 사양에 대한 정보를 알 수 있다.

```
#include <sys/utsname.h>
int uname(struct utsname *name);
```

```
#include <unistd.h>
int gethostname(char *name, size_t namelen);
```

- `uname` 함수에서 사용하는 `struct utsname` 구조체 자료형은 `sys/utsname.h` 파일에 다음과 같이 선언되어있다.

```
struct utsname {
    char sysname[SYS_NMLN];    // OS 종류
    char nodename[SYS_NMLN];   // 호스트 명
    char release[SYS_NMLN];    // 커널 릴리즈 번호
    char version[SYS_NMLN];    // 커널 빌드 정보
    char machine[SYS_NMLN];    // 하드웨어 사양
};
```

다음은 현재 동작중인 호스트 시스템의 정보를 확인하는 프로그램이다. 빈 칸을 완성하고 실행해봅시다.

소스코드 (ex05.c)

```
#include <sys/utsname.h>
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    struct utsname mysystem;
    char myhostname[256];

    if(gethostname(myhostname,255)!=0) {
        perror("gethostname");
        exit(1);
    }
    printf("gethostname = %s\n", myhostname);

    printf("OS 종류 = %s\n", mysystem.sysname);
    printf("hostname = %s\n", mysystem.nodename);
    printf("release = %s\n", mysystem.release);
    printf("version = %s\n", mysystem.version);
    printf("hardware = %s\n", mysystem.machine);
    return 0;
}
```

시간 정보

■ time(), gettimeofday() 함수

- 프로그램을 작성하다 보면 때때로 날짜 및 시간 정보가 필요한 경우가 있다. 예를 들어 실행 시간을 로그 파일에 기록하거나, 프로그램의 실행 시간을 측정해야 하는 경우가 그 예라 할 수 있다. 리눅스에서는 현재 시간을 확인하는 함수로 다음과 같은 함수를 제공한다.

```
#include <sys/time.h>
#include <unistd.h>

time_t time(time_t *tloc);
int gettimeofday(struct timeval *tv, struct timezone *tz);
```

- time() 함수는 초 단위의 시간으로, gettimeofday() 함수는 마이크로 초 단위의 시간까지 확인할 수 있는 함수로 여기에서 시간의 기준은 1970년 1월 1일 0시를 기준으로 한 시간이다. 즉, time() 함수의 수행 결과가 125405 라면 1970년 1월 1일 0시로부터 125405 초가 흘렀다는 의미이다. gettimeofday() 함수에서 사용하는 struct timeval은 time.h 에 다음과 같이 정의되어 있다.

```
struct timeval {
    long tv_sec;    /* 초 */
    long tv_usec;   /* 마이크로 초 */
};
```

다음 ex06.c는 time() 함수와 gettimeofday() 함수를 이용하여 현재 시간을 확인하는 프로그램이다.

```
#include <stdio.h>
#include <sys/time.h>
#include <unistd.h>

int main(void)
{
    time_t cur_time;
    struct timeval cur_gettimeofday;

    time(&cur_time);
    printf("time : current seconds = %d\n", cur_time);

    gettimeofday(&cur_gettimeofday, NULL);
    printf("current seconds = %d, micro seconds = %d\n", \
          cur_gettimeofday.tv_sec, cur_gettimeofday.tv_usec);
    return 0;
}
```

실행 결과에서 알 수 있듯이 time() 과 gettimeofday() 함수로부터 알 수 있는 시간 정보는 사람들이 사용하는 연, 월, 일, 시, 분, 초 단위의 시간 정보가 아니라 시간 확인이 불편하다. 이런 형식으로 시간 정보를 표현해야 하는 경우에는 ctime(), localtime(), strftime() 등의 라이브러리 함수를 이용하여 시간 정보를 변환하여 사용하면 된다. 이 시간 변환 함수에 대해서는 온라인 매뉴얼을 통해 자세한 사용 방법을 익히고 실습해보기로 하자.

[man 3 ctime](#)
[man 3 localtime](#)
[man 3 strftime](#)

시간 정보

■ times()함수

- 시스템의 시간을 확인해야 하는 경우도 있지만 때로는 프로그램의 실행 시간, 즉 프로세스의 시간 정보를 필요로 하는 경우도 있다. 이런 경우 사용할 수 있는 함수는 다음과 같다.

```
#include <sys/times.h>
clock_t times(struct tms *buf);
```

- struct tms는 /usr/include/sys/times.h에 다음과 같이 정의 되어 있다:

```
struct tms {
    clock_t tms_utime;    /* 사용자 모드의 시간 */
    clock_t tms_stime;    /* 시스템 모드의 시간 */
    clock_t tms_cutime;   /* 자식 프로세스의 사용자 모드 시간 */
    clock_t tms_cstime;   /* 자식 프로세스의 시스템 모드 시간 */
};
```

클럭틱 (clock tick): 컴퓨터를 구성하고 있는 수정 진동자가 일정한 간격으로 동작하고 있는 횟수를 클럭 틱 이라 한다. 이 클럭 틱이 발생될 때마다 CPU 가 명령을 실행하게 되므로, 1 초에 얼마나 많이 동작되는지 횟수를 헤아려 컴퓨터의 속도를 나타내기도 한다. 시스템의 시간도 이 클럭 틱을 이용해 시간 정보를 표현한다. sysconf(_SC_CLK_TCK) 함수를 이용하면 현재 사용하는 시스템의 클럭틱을 알 수 있다.

- times() 함수에서는 프로세스의 시간을 사용자 모드와 시스템 모드에서 측정하고 실지 수행시간을 리턴 한다. 이 때 시간 단위는 시스템 부팅 후 계산된 클럭 틱(clock tick)의 값으로 나타낸다.

다음 ex07.c은 Clock 수로 나오는 실행 결과를 초 단위로 나오도록 수정해보자.

```
...

int main(void)
{
    int i;
    time_t c_time;

    clock_t oldtime, newtime;
    struct tms oldtms, newtms;

    if((oldtime=times(&oldtms))== -1) {
        perror("old times");
        exit(1);
    }
    for(i=1; i<=99999999; i++)
        time(&c_time);
    if((newtime=times(&newtms))== -1) {
        perror("new times");
        exit(1);
    }
    printf("Real Time : %d clocks\n", newtime-oldtime);
    printf("User mode Time : %d clocks\n", newtms.tms_utime-oldtms.tms_utime);
    printf("System mode Time : %d clocks\n", newtms.tms_stime-oldtms.tms_stime);
    return 0;
}
```

자원 제한치 확인 및 변경

■ getrlimit(), setrlimit()

- 리눅스 시스템에서 사용되는 자원에는 제한 치가 설정되어 있다. 예를 들어 파일의 크기나 CPU 시간과 같은 시스템 자원에 설정된 제한된 값을 말한다. 리눅스에서 시스템 자원에 설정된 제한치를 확인하거나 변경하는 함수는 다음과 같다.

```
#include <sys/resource.h>

int getrlimit(int resource, struct rlimit *r_limit);
int setrlimit(int resource, const struct rlimit *r_limit);

struct rlimit
{
    rlim_t  rlim_cur;    // 현재 설정된 소프트 제한 값
    rlim_t  rlim_max;    // 절대적 제한치
};
```

- `getrlimit()` 함수는 시스템 자원의 제한치를 확인할 때, `setrlimit()`는 제한치를 변경할 때 사용하는 함수로 각 구조체 자료형 `struct rlimit`에 그 값을 읽어오거나 변경할 값을 대입한다. 구조체 자료형 중 `rlimit_cur` 멤버에는 일반적으로 권고하는 소프트 권고 치를, `rlimit_max` 멤버에는 하드 제한치를 설정할 수 있다.
- 첫 번째 인수에 나타낼 수 있는 리소스의 종류는 다음과 같다.

자원 제한치 확인 및 변경

Resource 종류	의미
RLIMIT_CPU	초 단위의 cpu 시간
RLIMIT_FSIZE	최대 파일 크기
RLIMIT_DATA	최대 데이터 크기
RLIMIT_STACK	최대 스택 크기
RLIMIT_CORE	최대 코어 파일 크기
RLIMIT_NPROC	최대 프로세스의 수
RLIMIT_NOFILE	동시에 열 수 있는 최대 파일의 수
RLIMIT_MEMLOCK	Lock된 기억 공간의 최대 크기
RLIMIT_AS	가상 주소 공간의 제한 값

다음 ex08.c 를 통해 시스템 자원의 현재 소프트 제한치와 하드 제한치를 살펴보고 소프트 제한치를 적용해보자.

```
...

void openfile_test(void);

int main(void)
{
    struct rlimit mylimit;

    getrlimit(RLIMIT_NPROC, &mylimit);
    printf("Current Number of Process : softlimit=%d, hardlimit=%d\n", \
        mylimit.rlim_cur, mylimit.rlim_max);

    getrlimit(RLIMIT_NOFILE, &mylimit);
    printf("\nCurrent Number of File : softlimit=%d, hardlimit=%d\n", \
        mylimit.rlim_cur, mylimit.rlim_max);

    mylimit.rlim_cur = 5;
    setrlimit(RLIMIT_NOFILE, &mylimit);
    openfile_test();
    return 0;
}
```

```
void openfile_test(void)
{
    char *filename[6] = { "test1", "test2", "test3", "test4", "test5",
        "test6" };
    FILE *fp;
    int i;

    for(i=0; i<6; i++)
    {
        if((fp=fopen(filename[i], "w"))==NULL) {
            perror(filename[i]);
            exit(1);
        }
    }
    return;
}
```

〈실습〉

- 프로그램 수행 시 입력되는 데이터들을 사용자의 홈 디렉토리에 파일로 저장하려고 한다. 이때 파일명은 해당 연도4자리와 월일 각 2자리씩 모두 8자로 구성할 예정이다. 해당 프로그램을 작성하시오